

```
import random

class AsteriaConfig:

    def __init__(self, robot_name, max_speed):

        self.robot_name = robot_name

        self.wheel_radius = 20

        self.max_speed = max_speed

        self.wheel_base = 30

        self.battery = 100

        self.is_moving= False

        self.distance= None

        self.temperature = None

        self.braking = False

        self.direction = None

        self.distance_right = None

        self.distance_left = None

        self.distance_rear = None

        self.distance_history = []

    def show_config(self):

        print(f"Robot name: {self.robot_name}")

        print(f"Wheel radius: {self.wheel_radius}")

        print(f"Maximum speed: {self.max_speed}")

        print(f"Wheel Base: {self.wheel_base}")

        print(f"Battery percentage is {self.battery}")

        print(f"Robot is:{self.is_moving}")

        print(f"Robot state of motion {self.direction}")

    def change_speed(self,new_speed):
```

```
self.max_speed = new_speed

if self.max_speed > 0:
    self.is_moving = True
else:
    self.is_moving = False

def stop_robot(self):
    self.max_speed = 0
    self.is_moving = False
    print(f"{self.robot_name} has stopped")

def consume_power(self, amount):
    self.battery -= amount

    if self.battery > 0:
        return self.battery
    elif self.battery <= 0:
        self.battery = 0
        print("Battery Depleted")
        return 0
    else:
        print("Battery Depleted")
        return 0

def update_distance(self, new_distance):
    self.distance = new_distance

def update_distance_right(self, new_distance_right):
    self.distance_right = new_distance_right

def update_distance_left(self, new_distance_left):
    self.distance_left = new_distance_left
```

```
def update_distance_rear(self,new_distance_rear):
```

```
    self.distance_rear = new_distance_rear
```

```
def can_move_forward(self):
```

```
    if self.distance is None:
```

```
        print("Front sensor data not available")
```

```
        return False
```

```
    elif self.distance>10:
```

```
        return True
```

```
    else:
```

```
        return False
```

```
def can_turn_right(self):
```

```
    if self.distance_right is None:
```

```
        print("Right sensor data not available")
```

```
        return False
```

```
    if self.distance_right>10:
```

```
        return True
```

```
    else:
```

```
        return False
```

```
def can_turn_left(self):
```

```
    if self.distance_left is None:
```

```
        print("Left sensor data not available")
```

```
        return False
```

```
    elif self.distance_left>10:
```

```
        return True
```

```
    else:
```

```
        return False
```

```
def can_reverse(self):  
    if self.distance_rear is None:  
        print("Rear sensor data not available")  
        return False  
    if self.distance_rear>10:  
        return True  
    else:  
        return False
```

```
def brakes(self):  
    if self.distance is None:  
        self.braking = False  
        return("Brakes not enabled")  
    if self.distance<10:  
        self.braking = True  
        self.stop_robot()  
    else:  
        self.braking = False
```

```
def move_forward(self):  
    if self.distance is None:  
        print("Front sensor data not available")  
    elif self.can_move_forward() is True:  
        self.direction = "Forward"  
        self.max_speed = 20  
        print("Asteria is moving Forward")
```

```
def turn_right(self):  
    if self.distance_right is None:  
        print("Right sensor data not available")  
    elif self.can_turn_right() is True:  
        self.direction = "Right"  
        self.max_speed= 20  
        print("Asteria is turning Right")  
    else:  
        self.braking = True  
        self.stop_robot()  
        print("Asteria cannot turn Right")  
  
def turn_left(self):  
    if self.distance_left is None:  
        print("Left sensor data not available")  
    elif self.can_turn_left() is True:  
        self.direction = "Left"  
        self.max_speed= 20  
        print("Asteria is turning Left")  
    else:  
        self.braking = True  
        self.stop_robot()  
        print("Asteria cannot turn Left")  
  
def reverse(self):  
    if self.distance_rear is None:  
        print("Rear sensor data not available")  
    elif self.can_reverse():
```

```

        self.direction = "Rear"

        self.max_speed= 20

        print("Asteria is moving backward")
    else:

        self.braking = True

        self.stop_robot()

        print("Asteria cannot move backward")

def reverse_check_direction(self):

    if self.can_reverse():

        self.reverse()

    if self.distance>25:

        self.braking = True

        self.stop_robot()

    else:

        print("Only rear path clear")


def decide_direction(self):

    if self.can_turn_left() and self.can_turn_right():

        if self.distance_left>self.distance_right:

            self.turn_left()

        elif self.distance_left == self.distance_right:

            self.turn_left()

    elif self.can_turn_left():

        self.turn_left()

    elif self.can_turn_right():

        self.turn_right()

```

```
else:  
    self.reverse_check_direction()
```

```
def avoid_obstacles(self):  
    if self.distance is None:  
        print("Front sensor data not available")  
    elif self.distance<10:  
        self.decide_direction()  
    else:  
        self.move_forward()  
def remember_distance(self):  
    self.distance_history.append(self.distance)
```

```
class Environment:  
    def __init__(self):  
        self.temperature = 25  
        self.luminosity = 80  
        self.humidity = 50  
        self.front_distance = 100  
        self.rear_distance = 100  
        self.right_distance = 100  
        self.left_distance = 100  
    def update_front_distance(self,new_front_distance):  
        self.front_distance = new_front_distance  
    def update_rear_distance(self,new_rear_distance):  
        self.rear_distance = new_rear_distance
```

```

def update_right_distance(self,new_right_distance):
    self.right_distance = new_right_distance
def update_left_distance(self,new_left_distance):
    self.left_distance= new_left_distance
def update_temperature(self,new_temperature):
    self.temperature = new_temperature
def update_humidity(self,new_humidity):
    self.humidity = new_humidity
def update_luminosity(self,new_luminosity):
    self.luminosity = new_luminosity
def display_environment(self):
    print(f"_____CURRENT ENVIRONMENT_____\nFront Distance : {self.front_distance}
cm\nRear Distance : {self.rear_distance} cm\nLeft Distance : {self.left_distance} cm\nRight
Distance : {self.right_distance} cm\nTemperature : {self.temperature} K\nHumidity :
{self.humidity} %\nLuminosity : {self.luminosity} lux")
def reset_environment(self):
    self.temperature = None
    self.luminosity = None
    self.humidity = None
    self.front_distance = None
    self.rear_distance = None
    self.right_distance = None
    self.left_distance = None
class EnvironmentAnalyzer:
    def __init__(self,environment):
        self.environment = environment
    def analyze_temperature(self):

```



```
temperature =self.environment.temperature

if temperature is None:

    return "No readings available"

elif temperature<30:

    return"Deep space like"

elif temperature<150:

    return"Comet like"

elif temperature<273:

    return"Mars like"

elif temperature<310:

    return"Earth like"

elif temperature<330:

    return"Warm Earth like"

elif temperature<800:

    return"Venus like"

elif temperature<1000:

    return "Mercury Day-side like"

elif temperature<5800:

    return"Solar Surface like"

elif temperature<9000000:

    return"Solar Corona like"

elif temperature<15000000:

    return"Solar Core like"

else:

    return"Temperature exceeds known limit"
```

```
def analyze_luminosity(self):  
    luminosity = self.environment.luminosity  
    if luminosity is None:  
        return "No readings available"  
    elif luminosity<=0:  
        return "Deep cave like"  
    elif luminosity<1:  
        return "Moonless night in the open sky"  
    elif luminosity<10:  
        return "Full moon night"  
    elif luminosity<100:  
        return "Twilight"  
    elif luminosity<500:  
        return "Laboratory"  
    elif luminosity<10000:  
        return "Overcast Earth"  
    elif luminosity<100000:  
        return "Earth at noon"  
    elif luminosity<500000:  
        return "Bright desert"  
    elif luminosity<1000000:  
        return "Mercury Surface"  
    else:  
        return "Near a bright star"  
def analyze_humidity(self):
```

```
humidity = self.environment.humidity

if humidity is None:

    return "Readings not available"

elif humidity<=0:

    return "Outer Space"

elif humidity<5:

    return "Interplanetary Space"

elif humidity<15:

    return "Atacama Desert"

elif humidity<30:

    return "Sahara Desert"

elif humidity<45:

    return "Heated indoor environment"

elif humidity<60:

    return "Typical Earth indoor environment"

elif humidity<75:

    return "Tropical Climate"

elif humidity<90:

    return "Rainforest"

elif humidity<100:

    return "Dense fog"

else:

    return "Sensor malfunction"

def analyze_front_distance(self):

    front_distance= self.environment.front_distance

    if front_distance is None:
```

```

        return "No reading available"
    elif front_distance < 0:
        return "Invalid reading"
    elif front_distance < 5:
        return "Collision Imminent"
    elif front_distance < 15:
        return "Extremely Close"
    elif front_distance < 30:
        return "Very Close"
    elif front_distance < 60:
        return "Close"
    elif front_distance < 100:
        return "Moderate Distance"
    elif front_distance < 300:
        return "Clear Path"
    elif front_distance < 1000:
        return "Open Area"
    else:
        return "Wide Open Area"

def analyze_rear_distance(self):
    rear_distance = self.environment.rear_distance
    if rear_distance is None:
        return "No reading available"
    elif rear_distance < 0:
        return "Invalid reading"
    elif rear_distance < 5:

```

```
        return "Collision Imminent"
    elif rear_distance < 15:
        return "Extremely Close"
    elif rear_distance < 30:
        return "Very Close"
    elif rear_distance < 60:
        return "Close"
    elif rear_distance < 100:
        return "Moderate Distance"
    elif rear_distance < 300:
        return "Clear Path"
    elif rear_distance < 1000:
        return "Open Area"
    else:
        return "Wide Open Area"

def analyze_right_distance(self):
    right_distance = self.environment.right_distance
    if right_distance is None:
        return "No reading available"
    elif right_distance < 0:
        return "Invalid reading"
    elif right_distance < 5:
        return "Collision Imminent"
    elif right_distance < 15:
        return "Extremely Close"
    elif right_distance < 30:
```

```
        return "Very Close"
    elif right_distance < 60:
        return "Close"
    elif right_distance < 100:
        return "Moderate Distance"
    elif right_distance < 300:
        return "Clear Path"
    elif right_distance < 1000:
        return "Open Area"
    else:
        return "Wide Open Area"
def analyze_left_distance(self):
    left_distance = self.environment.left_distance
    if left_distance is None:
        return "No reading available"
    elif left_distance < 0:
        return "Invalid reading"
    elif left_distance < 5:
        return "Collision Imminent"
    elif left_distance < 15:
        return "Extremely Close"
    elif left_distance < 30:
        return "Very Close"
    elif left_distance < 60:
        return "Close"
    elif left_distance < 100:
```

```

        return "Moderate Distance"
elif left_distance < 300:
    return "Clear Path"
elif left_distance < 1000:
    return "Open Area"
else:
    return "Wide Open Area"
def overall_status(self):
    print("Analyzer temperature", self.environment.temperature)
    print("analyzer front", self.environment.front_distance)
    warnings = []
    if self.environment.temperature is not None:
        if self.environment.temperature > 120:
            warnings.append("Critical thermal stress: extreme heat")
        elif self.environment.temperature < -150:
            warnings.append("Critical thermal stress: extreme cold")
        elif self.environment.temperature > 80:
            warnings.append("High temperature detected")
        elif self.environment.temperature < -100:
            warnings.append("Low temperature detected")
    if self.environment.front_distance is not None:
        if self.environment.front_distance < 5:
            warnings.append("Immediate obstacle ahead")
        elif self.environment.front_distance < 20:
            warnings.append("Object detected ahead")
    if self.environment.rear_distance is not None:

```

```

        if self.environment.rear_distance<5:
            warnings.append("Immediate obstacle behind")
    if self.environment.left_distance is not None:
        if self.environment.left_distance<5:
            warnings.append("Obstacle detected on left")
    if self.environment.right_distance is not None:
        if self.environment.right_distance<5:
            warnings.append("Obstacle detected on right")
    if len(warnings) == 0:
        return{
            "Status": "SAFE",
            "Warnings": []
        }
    elif len(warnings)==1:
        return{
            "Status": "RISKY",
            "Warnings": warnings
        }
    else:
        return{
            "Status": "DANGER",
            "Warnings": warnings
        }

```

```

class EnvironmentSnapshot:
    def __init__(self,environment):

```



```
self.front_distance = environment.front_distance
self.rear_distance = environment.rear_distance
self.right_distance = environment.right_distance
self.left_distance = environment.left_distance
self.temperature = environment.temperature
self.humidity = environment.humidity
self.luminosity = environment.luminosity
```

```
class Action:
```

```
    def __init__(self):
        self.action_name= None
        self.execution_time=None
        self.successful=None
        self.timestamp=None
        self.battery_used=None
        self.direction=None
        self.reason = None
        self.outcome = None
```

```
class Experience:
```

```
    def __init__(self):
        self.environment_before= None
        self.environment_after= None
        self.action=None
        self.outcome=None
        self.timestamp=None
```

```
class Memory:

    def __init__(self):

        self.last_environment = None

        self.previous_environment = None

        self.last_action = None

        self.previous_action = None

        self.last_outcome=None

        self.last_timestamp=None

        self.experience_log = []

    def remember_environment(self,environment):

        self.previous_environment=self.last_environment

        snapshot = EnvironmentSnapshot(environment)

        self.last_environment=snapshot

    def remember_action(self,action):

        self.previous_action=self.last_action

        self.last_action= action

    def save_experience(self):

        experience=Experience()

        experience.environment_before=self.previous_environment

        experience.environment_after=self.last_environment

        experience.action=self.last_action

        experience.outcome=self.last_outcome

        experience.timestamp=self.last_timestamp

        self.experience_log.append(experience)

    def show_last_experience(self):
```

```

if self.save_experience.experience is None:

    return("No experiences available")

else:

    print(f"____Previous Experience____\n Initial conditions:{self.previous_environment}\n
Action taken:{self.last_action}\n Final conditions:{self.last_environment}")

def reset_experience(self):

    self.previous_environment = None

    self.last_environment =None

    self.last_action= None

class AsteriaBrain:

def __init__(self,environment_analyzer,robot,mission_controller,memory):

    self.environment_analyzer=environment_analyzer

    self.robot = robot

    self.mission_controller=mission_controller

    self.memory=memory

def evaluate_situation(self):

    status = self.environment_analyzer.overall_status()


    if status["Status"] == "SAFE":

        return "Continue Exploration"


    elif status["Status"] == "RISKY":

        return "Proceed with caution"


    elif status["Status"] == "DANGER":

        return "Stop and reassess"

```

```

else:
    return "Stop and reassess"

def take_action(self):
    decision = self.evaluate_situation()
    if decision=="Continue Exploration":
        self.robot.move_forward()
    elif decision=="Proceed with caution":
        self.robot.change_speed(10)
    elif decision=="Stop and reassess":
        self.robot.stop_robot()

def make_decision(self,status,mission):
    warnings=status["Warnings"]
    if mission == "Recharge":
        return"Return to charging station"
    elif mission=="Avoid Danger":
        return self.check_direction()
    elif mission=="Proceed Carefully":
        if status["Status"]=="RISKY":
            if "Obstacle detected on right" in warnings:
                return self.check_direction()
            elif "Object detected ahead" in warnings:
                return self.check_direction()
        else:
            return "Proceed Carefully"
    elif mission=="Explore":

```

```
if status["Status"]=="SAFE":
    return"Move forward and continue exploration"
elif status["Status"]=="RISKY":
    if "Object detected ahead" in warnings:
        return self.check_direction()
    else:
        return"Proceed carefully"
elif status["Status"]=="DANGER":
    if "Immediate obstacle ahead" in warnings:
        return self.check_direction()
    else:
        return"Stop and reassess environment"
else:
    return "Stop and wait"
```

```
def check_direction(self):
    if self.robot.can_turn_left():
        return"Turn left"
    elif self.robot.can_turn_right():
        return"Turn right"
    elif self.robot.can_reverse():
        return"Reverse"
    else:
        return"Stop: No safe direction available"
def execute_decision(self,decision):
```

```

if decision=="Turn left":
    self.robot.turn_left()
elif decision=="Turn right":
    self.robot.turn_right()
elif decision=="Reverse":
    self.robot.reverse_check_direction()
elif decision=="Stop: No safe direction available":
    self.robot.stop_robot()
elif decision=="Move forward and continue exploration":
    self.robot.move_forward()
elif decision=="Proceed carefully":
    self.robot.change_speed(10)
def run_cycle(self):

    status = self.environment_analyzer.overall_status()
    mission=self.mission_controller.choose_mission(status,self.robot.battery)
    decision = self.make_decision(status,mission)
    self.execute_decision(decision)


    decision= self.make_decision(status,mission)
    self.execute_decision(decision)
    if self.should_save_experience(status,decision):
        self.memory.save_experience()
    return{

```

```

        "Mission": mission,
        "Status": status["Status"],
        "Decision": decision,
        "Battery": self.robot.battery
    }

def run(self,cycles):
    for _ in range(cycles):
        self.run_cycle()

def should_save_experience(self,status,decision):
    if status["Status"]=="DANGER":
        return True

    elif status["Status"]=="RISKY":
        return True

    elif decision=="Stop and reassess":
        return True

    else:
        return False

```

```

class Mission:
    def __init__(self):
        self.name = None
        self.status= "Not Started"
        self.completed= False
        self.priority = "Normal"
        self.progress = None

    def start_mission(self):

```

```

        self.status= "Running"

        self.completed= False

    def complete_mission(self):

        self.status= "Completed"

        self.completed= True

    def abort_mission(self):

        pass

class MissionController:

    def __init__(self):

        self.current_mission= None

    def choose_mission(self,status,battery):

        if battery<20:

            self.current_mission="Recharge"

        elif status["Status"]=="DANGER":

            self.current_mission="Avoid Danger"

        elif status["Status"]=="RISKY":

            self.current_mission="Proceed Carefully"

        elif status["Status"]=="SAFE":

            self.current_mission="Explore"

        else:

            self.current_mission="Standby"

        return self.current_mission

    def update_mission(self,status,battery):

        if self.current_mission is None:

            self.choose_mission(status,battery)

        if self.current_mission=="Recharge":

```



```

        if battery==100:
            self.current_mission.complete_mission(status,battery)
        if self.current_mission.name == "Avoid Danger":
            if status["Status"]=="SAFE":
                self.current_mission.complete_mission(status,battery)
            elif self.current_mission=="Explore":
                if battery<20 or status["Status"]=="DANGER":
                    self.current_mission= self.choose_mission(status,battery)
class Simulation:
    def __init__(self,environment,brain):
        self.environment=environment
        self.brain=brain
        self.cycle=0
    def generate_event(self):
        event = random.randint(1,100)
        if event<=70:
            pass
        elif event<=85:
            self.environment.front_distance=random.randint(10,30)
        elif event<=95:
            self.environment.temperature=random.randint(80,100)
        else:
            self.environment.luminosity=random.randint(0,5)
    def next_cycle(self):
        self.cycle +=1
        self.generate_event()

```

```
self.environment.temperature+=random.randint(-5,5)

self.environment.front_distance+= random.randint(-2,2)
if self.environment.front_distance<0:
    self.environment.front_distance=0
self.environment.luminosity+=random.uniform(-0.5,0.5)
if self.environment.luminosity<0:
    self.environment.luminosity=0
self.environment.humidity+=random.randint(-3,3)
if self.environment.humidity<0:
    self.environment.humidity = 0
self.environment.rear_distance+=random.randint(-2,2)
if self.environment.rear_distance<0:
    self.environment.rear_distance=0
self.environment.right_distance+=random.randint(-2,2)
if self.environment.right_distance<0:
    self.environment.right_distance=0
self.environment.left_distance+=random.randint(-2,2)
if self.environment.left_distance<0:
    self.environment.left_distance=0
self.brain.robot.update_distance(self.environment.front_distance)
self.brain.robot.update_distance_left(self.environment.left_distance)
self.brain.robot.update_distance_right(self.environment.right_distance)
self.brain.robot.update_distance_rear(self.environment.rear_distance)
result=self.brain.run_cycle()
self.show_cycle(result)
```

```

def run_simulation(self,cycles):

    for _ in range(cycles):

        self.next_cycle()


def show_cycle(self, result):

    print("=" * 55)

    print("          ASTERIA 1.0")

    print("=" * 55)


    print(f"Cycle    : {self.cycle}")


    print("\nMISSION")

    print("-" * 25)

    print(f"Mission   : {result['Mission']}")

    print(f"Status    : {result['Status']}")

    print(f"Decision  : {result['Decision']}")


    print("\nROBOT")

    print("-" * 25)

    print(f"Battery   : {self.brain.robot.battery}%")


    print("\nENVIRONMENT")

    print("-" * 25)

    print(f"Temperature: {self.environment.temperature} °C")

    print(f"Humidity   : {self.environment.humidity} %")

    print(f"Luminosity : {self.environment.luminosity}")

```

```
print(f"Front    : {self.environment.front_distance} m")
print(f"Left     : {self.environment.left_distance} m")
print(f"Right    : {self.environment.right_distance} m")
print(f"Rear     : {self.environment.rear_distance} m")
```

```
print("=" * 55)
```

```
class AsteriaTester:
```

```
    def __init__(self,simulation):
```

```
        self.simulation=simulation
```

```
    def test_safe_environment(self):
```

```
        self.simulation.environment.temperature = 25
```

```
        self.simulation.environment.front_distance = 30
```

```
        self.simulation.environment.left_distance = 30
```

```
        self.simulation.environment.right_distance = 30
```

```
        self.simulation.environment.rear_distance = 30
```

```
        self.simulation.environment.humidity = 40
```

```
        self.simulation.environment.luminosity = 3
```

```
        self.simulation.next_cycle()
```

```
    def test_low_battery(self):
```

```
        self.simulation.brain.robot.battery=15
```

```
        self.simulation.next_cycle()
```

```
    def test_obstacle_ahead(self):
```

```
        self.front_distance=2
```

```
    def run_all_tests(self):
```

```
        self.test_low_battery()
```

```
        self.test_obstacle_ahead()
asteria = AsteriaConfig("Asteria",100)
asteria.consume_power(40)
asteria.change_speed(150)
asteria.update_distance(10)
asteria.stop_robot()
asteria.show_config()
asteria.brakes()
asteria.move_forward()
environment=Environment()
environment_analyzer=EnvironmentAnalyzer(environment)
memory=Memory()
mission_controller=MissionController()

snapshot = EnvironmentSnapshot(environment)
robot=AsteriaConfig("Asteria",50)
brain=AsteriaBrain(environment_analyzer,robot,mission_controller,memory)
simulation=Simulation(environment,brain)
#simulation.run_simulation(100)
```